

第三章程程序的机器级表示V: 高级主题

教 师：邹翔宇

计算机科学与技术学院

哈尔滨工业大学（深圳）

章节要求

- 明确内存布局细节
- 掌握缓冲区溢出原理（概念题）
- 知道如何预防缓冲区溢出（简答题）
- 明确一段代码在大端序小端序下的输出
- 经典例题

主要内容

- 内存布局
- 缓冲区溢出
 - 安全隐患
 - 防护
- 联合

x86-64 Linux 内存布局

未按比例绘制

00007FFFFFFFH

8MB

Stack

Shared
Libraries

Heap

Data

Text

400000H

000000H

- 栈(Stack)
 - 运行时栈 (8MB limit)
 - 涉及局部变量
- 堆(Heap)
 - 按需动态分配
 - 时机:调用malloc(), calloc(), new()时
- 数据(Data)
 - 静态分配的内存中保存的数据
 - 全局变量、static变量、字符串常量
- 代码/共享库(Text / Shared Libraries)
 - 只读的可执行的机器指令

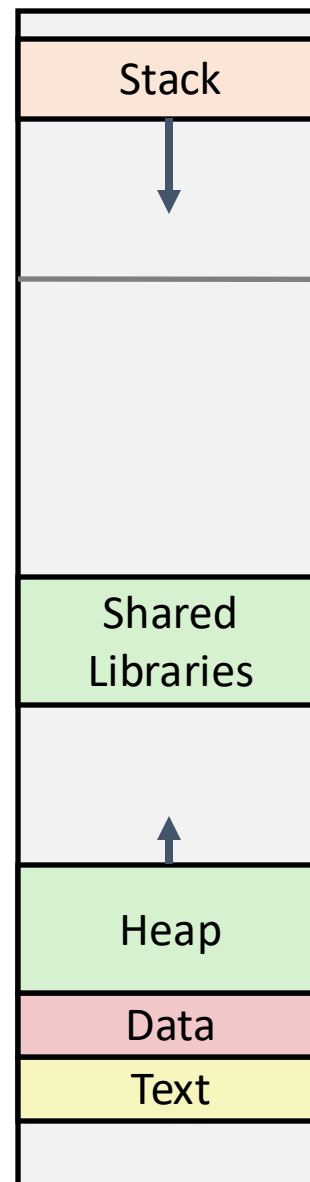
内存分配示例

```
char big_array[1L<<24]; /* 16 MB */
char huge_array[1L<<31]; /* 2 GB */

int global = 0;
int useless() { return 0; }
int main ()
{
    void *p1, *p2, *p3, *p4;
    int local = 0;
    p1 = malloc(1L << 28); /* 256 MB */
    p2 = malloc(1L << 8); /* 256 B */
    p3 = malloc(1L << 32); /* 4 GB */
    p4 = malloc(1L << 8); /* 256 B */
    /* Some print statements ... */
}
```

程序中各个部分都在哪里?

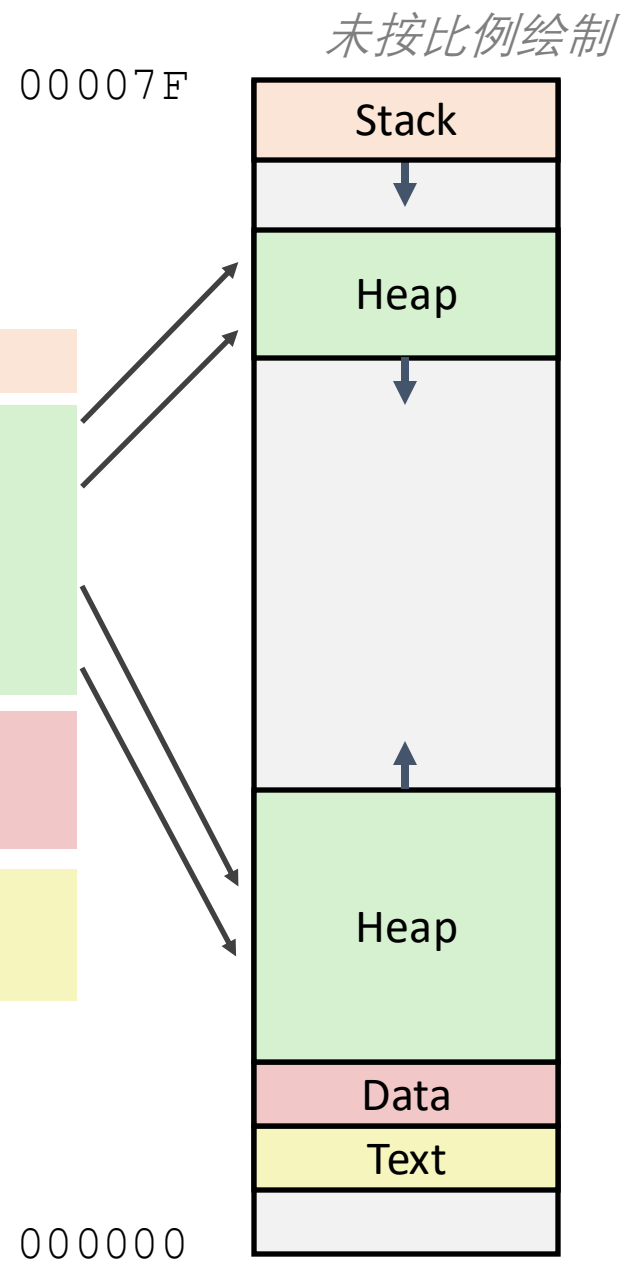
未按比例绘制



x86-64 例子的地址

地址范围 $\sim 2^{47}$

local	0x00007ffe4d3be87c
p1	0x00007f7262a1e010
p3	0x00007f7162a1d010
p4	0x000000008359d120
p2	0x000000008359d010
big_array	0x0000000080601060
huge_array	0x0000000000601060
main()	0x000000000040060c
useless()	0x0000000000400590



主要内容

- 内存布局
- 缓冲区溢出
 - 安全隐患
 - 防护
- 联合

内存引用的Bug示例

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;  
  
double fun(int i) {  
    volatile struct_t s;  
    s.d = 3.14;  
    s.a[i] = 1073741824; /* Possibly out of bounds */  
    return s.d;  
}
```

fun(0) → 3.14
fun(1) → 3.14
fun(2) → 3.1399998664856
fun(3) → 2.00000061035156
fun(4) → 3.14
fun(6) → Segmentation fault

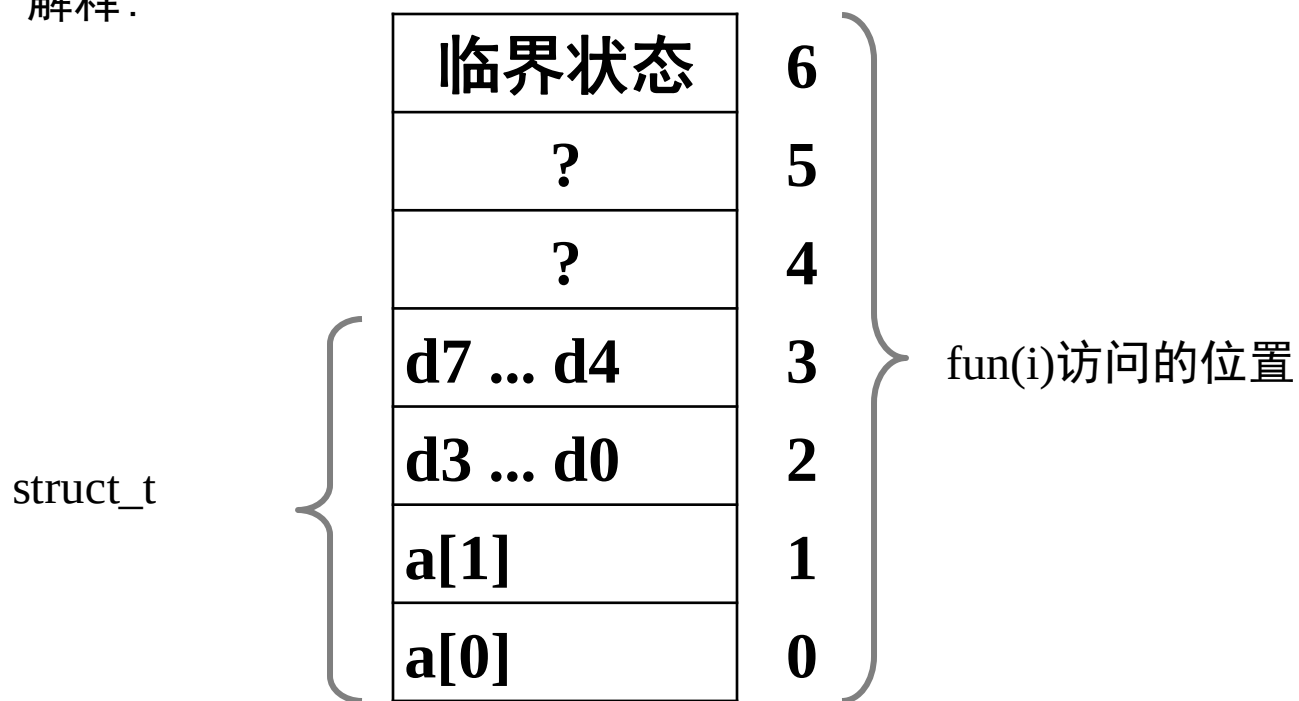
运行结果与系统有关

内存引用的Bug示例

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;
```

fun(0) ➔ 3.14
fun(1) ➔ 3.14
fun(2) ➔ 3.1399998664856
fun(3) ➔ 2.00000061035156
fun(4) ➔ 3.14
fun(6) ➔ Segmentation fault

解释:



这是个 **大** 问题

- 一般称为“缓冲区溢出”
 - 当超出数组分配的内存大小（范围）
- 为何是大问题？
 - 破坏性修改
 - 返回地址覆盖
- 更一般的形式
 - 字符串输入不检查长度
 - 特别是堆栈上的有界字符数组
 - 有时称为堆栈粉碎(stack smashing)

字符串库的代码

- Unix函数 `gets()` 的实现

```
/* Get string from stdin */
char *gets(char *dest){
    int c = getchar();
    char *p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

- 其他库函数也有类似问题

- **strcpy, strcat**: 任意长度字符串的拷贝
- **scanf, fscanf, sscanf**, 使用 **%s** 转换符时

存在安全隐患的缓冲区代码

```
/* Echo Line */  
void echo()  
{  
    char buf[4]; /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

问题：分配多大才足够？

```
void call_echo() {  
    echo();  
}
```

```
unix> ./bufdemo-nsp  
Type a string: 012345678901234567890123  
012345678901234567890123
```

```
unix> ./bufdemo-nsp  
Type a string: 0123456789012345678901234  
Segmentation Fault
```

缓冲区溢出的反汇编

echo:

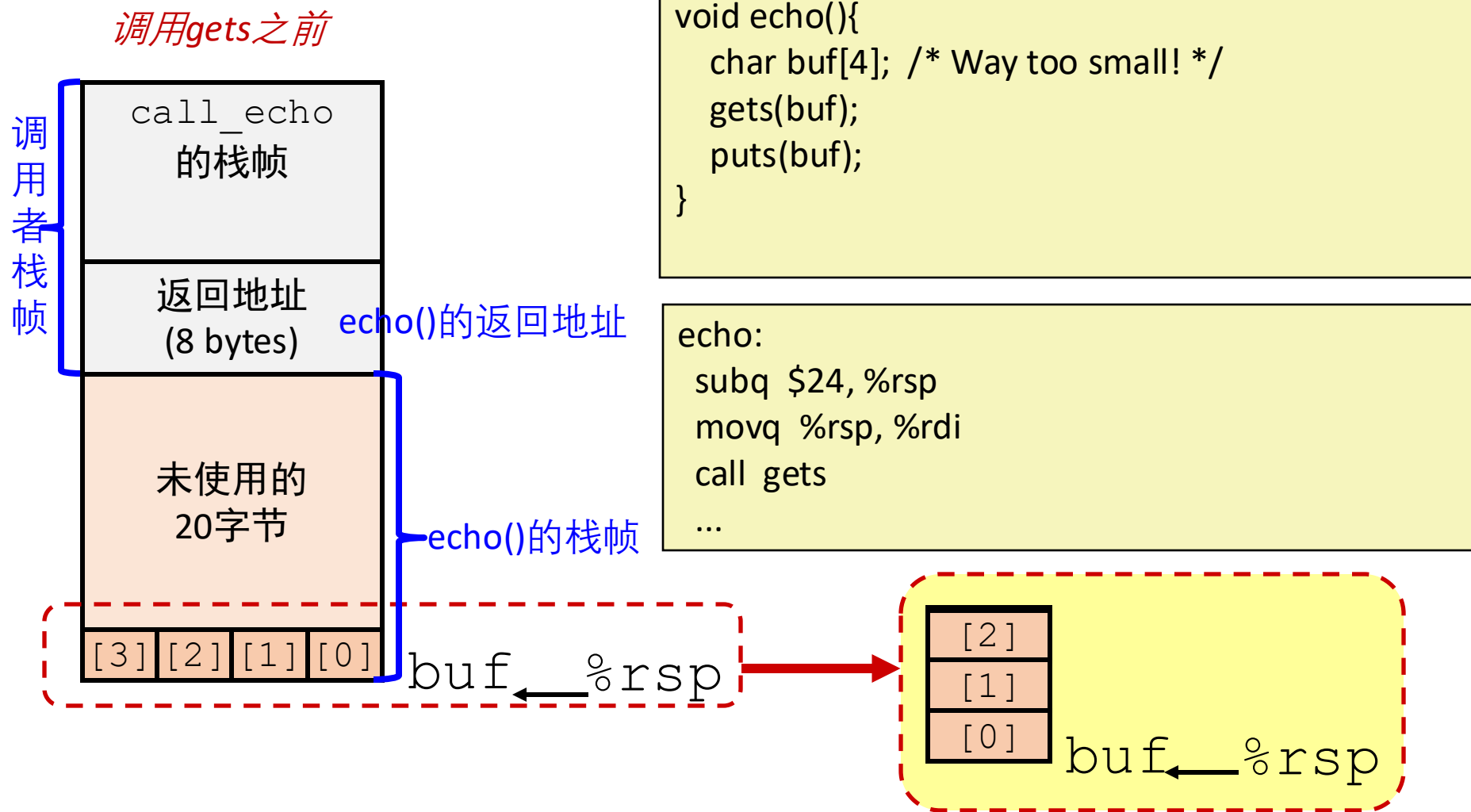
00000000004006cf <echo>:

4006cf:	48 83 ec 18	sub	\$0x18 , %rsp
4006d3:	48 89 e7	mov	%rsp,%rdi
4006d6:	e8 a5 ff ff ff	callq	400680 <gets>
4006db:	48 89 e7	mov	%rsp,%rdi
4006de:	e8 3d fe ff ff	callq	400520 <puts@plt>
4006e3:	48 83 c4 18	add	\$0x18,%rsp
4006e7:	c3	retq	

call_echo:

4006e8:	48 83 ec 08	sub	\$0x8,%rsp
4006ec:	b8 00 00 00 00	mov	\$0x0,%eax
4006f1:	e8 d9 ff ff ff	callq	4006cf <echo>
4006f6:	48 83 c4 08	add	\$0x8,%rsp
4006fa:	c3	retq	

缓冲区溢出的栈示例



缓冲区溢出的栈示例

调用gets之前



```
void echo(){  
    char buf[4];  
    gets(buf);  
    ...  
}
```

call_echo:

```
echo:  
    subq $24, %rsp  
    movq %rsp, %rdi  
    call gets  
    ...
```

```
...  
4006f1:    callq 4006cf <echo>  
4006f6:    add    $0x8,%rsp  
...
```

buf ← %rsp

缓冲区溢出的栈示例 #1

调用gets之后

call_echo 的栈帧			
00	00	00	00
00	40	06	f6
00	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

```
void echo(){  
    char buf[4];  
    gets(buf);  
    ...  
}
```

```
echo:  
    subq $24,%rsp  
    movq %rsp,%rdi  
    call gets  
    ...
```

call_echo:

```
...  
4006f1:    callq 4006cf <echo>  
4006f6:    add   $0x8,%rsp  
...
```

buf ← %rsp

缓冲区溢出，但没有破坏状态

```
unix> ./bufdemo-nsp  
Type a string: 01234567890123456789012  
01234567890123456789012
```


缓冲区溢出的栈示例 #2

调用gets之后

call_echo 的栈帧			
00	00	00	00
00	40	00	34
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ← %rsp

```
void echo(){
    char buf[4];
    gets(buf);
    ...
}
```

```
echo:
    subq $24,%rsp
    movq %rsp,%rdi
    call gets
    ...
```

```
call_echo:
    ...
4006f1:      callq 4006cf <echo>
4006f6:      add  $0x8,%rsp
    ...
```

溢出的缓冲区，
返回地址被破坏

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901234
Segmentation Fault
```

缓冲区溢出的栈示例 #3

调用gets之后

call_echo 的栈帧			
00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

```
void echo(){
    char buf[4];
    gets(buf);
    ...
}
```

```
echo:
    subq $24,%rsp
    movq %rsp,%rdi
    call gets
    ...
```

call_echo:

```
...
4006f1:    callq 4006cf <echo>
4006f6:    add    $0x8,%rsp
...
```

← %rsp

溢出的缓冲区,破坏了
返回地址, 但程序看
起来能工作

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901230123
45678901234567890123
```

缓冲区溢出的栈示例 #3 —— 解读

调用gets之后

call_echo 的栈帧			
00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ← %rsp

register_tm_clones:

```
...
400600:    mov    %rsp,%rbp
400603:    mov    %rax,%rdx
400606:    shr    $0x3f,%rdx
40060a:    add    %rdx,%rax
40060d:    sar    %rax
400610:    jne    400614
400612:    pop    %rbp
400613:    retq
```

返回到无关的代码

大多数情况不会修改临界状态，最终执行retq 返回主程序

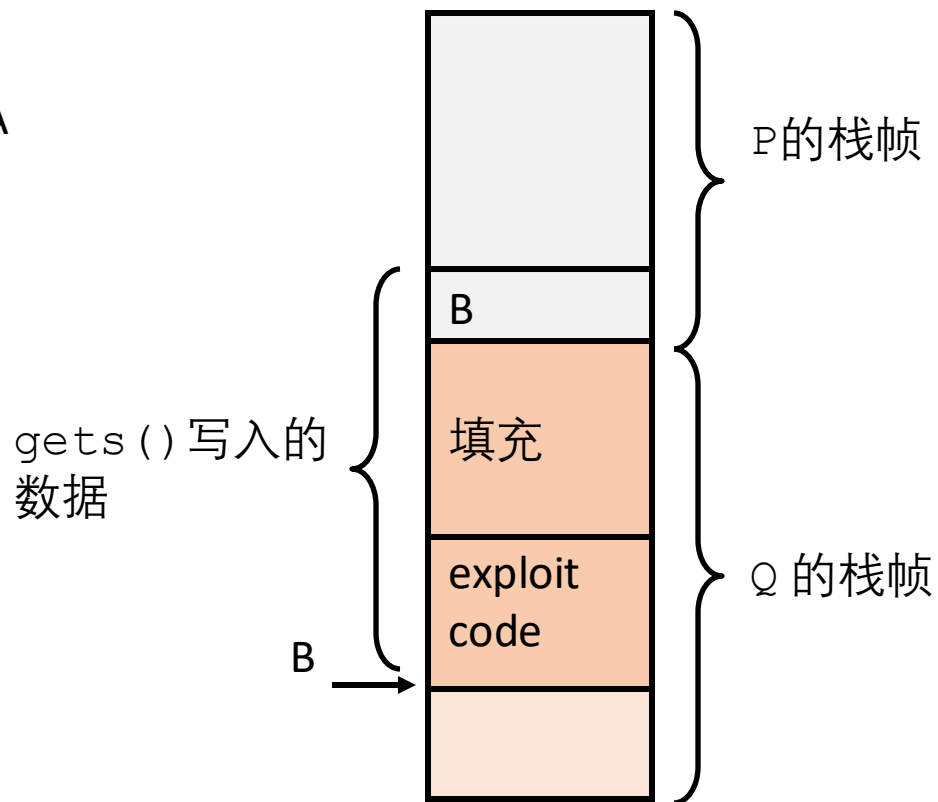
代码注入攻击(Code Injection Attacks)

```
void P(){  
    Q();  
    ...  
}
```

← 返回地址A

```
int Q() {  
    char buf[64];  
    gets(buf);  
    ...  
    return ...;  
}
```

调用gets()后的栈



- 输入字符串包含可执行代码的字节序列!
- 将返回地址 A 用缓冲区 B 的地址替换
- 当 Q 执行 ret 后, 将跳转到 B 处, 执行漏洞利用程序(exploit code)

基于缓冲区溢出的漏洞利用程序

- 缓冲区溢出错误允许远程机器在受害者机器上执行任意代码。
- 在程序中常见，令人不安
 - 程序员持续犯相同的错误
 - 最近的措施使这些攻击更加困难。
- 经典案例
 - 原始"互联网蠕虫"(Internet worm),1988
 - 即时通讯战争"IM wars",1999
 - Twilight hack on Wii, 2000s(不改动硬件，直接在Wii上运行自制程序)
- 在相应的实验中会学到一些技巧
 - 希望能说服你永远不要在程序中留下这样的漏洞！！

例子: 原始互联网蠕虫 (1988)

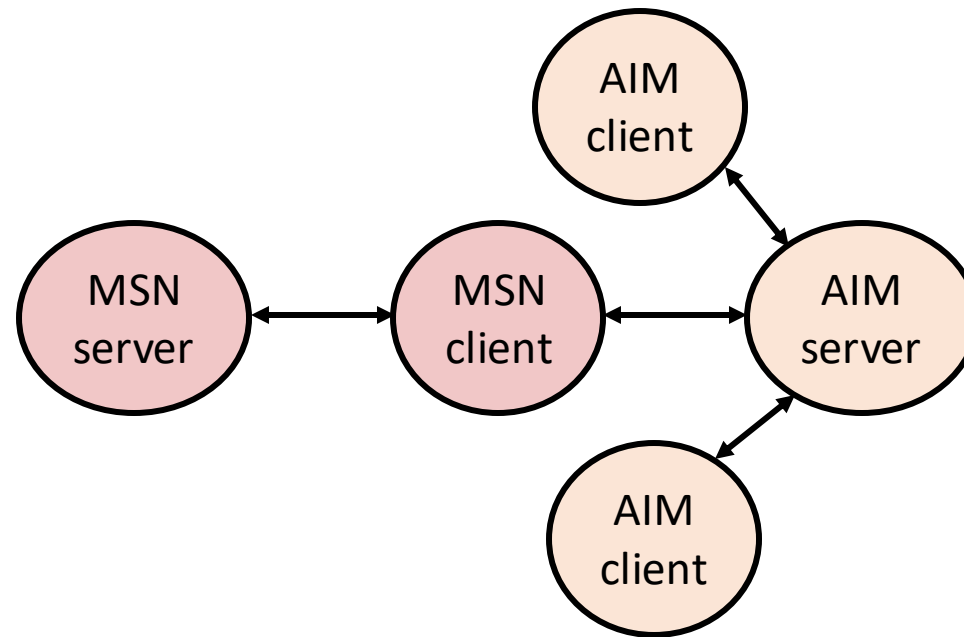
- 利用漏洞传播
 - 指服务器(finger server)的早期版本用`gets()` 读取客户机发来的参数:
 - **`finger droh@cs.cmu.edu`**
 - 蠕虫利用发送假参数的方法攻击指服务器:
 - **`finger "exploit-code padding new-return-address"`**
 - 利用程序:用直接和攻击者相连的TCP链接, 在受害者机器上执行根用户shell

例子: 原始互联网蠕虫 (1988)

- 一旦进到机器上, 就扫描其他机器攻击
- 几小时内侵入了大概6000台 (互联网机器总数的10%)
 - 参考*Comm. of the ACM*在1989年6月的文章
 - 年轻的蠕虫作者被起诉.....
 - 计算机安全应急响应组(Computer Emergency Response Team)成立, 现仍在CMU

例2:即时通讯战争

- 1999年7月
 - 微软发布了即时通讯系统MSN Messenger
 - Messenger 的客户端能获取流行的美国在线(American Online, AOL)即时通讯服务 (AIM) 服务器



例2:即时通讯战争 (续...)

- 1999年8月
 - AIM开始阻止微软登录, Messenger客户端无法再用AIM服务器
 - 微软和AOL 开始了即时通讯战争
 - AOL 变动服务器不允许Messenger客户端连接
 - 微软对客户进行更改以挫败AOL的变动
 - 至少有13个这样的小冲突
 - 真正发生的到底是什么?
 - AOL 在他们自己的AIM服务器中发现了缓冲区溢出的漏洞
 - 他们利用这个bug检测并阻塞微软: 漏洞利用客户端程序返回一个4字节的签名(在AIM客户端的某些位置存储的)到服务器
 - 当微软改变签名匹配程序时, AOL改变签名的位置

Date: Wed, 11 Aug 1999 11:30:57 -0700 (PDT)
From: Phil Bucking <philbucking@yahoo.com>
Subject: AOL exploiting buffer overrun bug in their own software!
To: rms@pharlap.com

Mr. Smith,

I am writing you because I have discovered something that I think you might find interesting because you are an Internet security expert with experience in this area. I have also tried to contact AOL but received no response.

I am a developer who has been working on a revolutionary new instant messaging client that should be released later this year.

...

It appears that the AIM client has a buffer overrun bug. By itself this might not be the end of the world, as MS surely has had its share. But AOL is now *exploiting their own buffer overrun bug* to help in its efforts to block MS Instant Messenger.

....

Since you have significant credibility with the press I hope that you can use this information to help inform people that behind AOL's friendly exterior they are nefariously compromising peoples' security.

Sincerely,
Phil Bucking
Founder, Bucking Consulting
philbucking@yahoo.com

后来确定这封电子邮件来源于微软内部!

PS: 蠕虫和病毒

- 蠕虫(Worm):程序
 - 可以自行运行
 - 可以将自己的完整版本传播到其他计算机上
- 病毒(Virus): 代码
 - 将自己添加到别的程序中
 - 不独立运行
- 两者通常都能在计算机之间传播并造成破坏。

缓冲区溢出漏洞非常普遍

The screenshot shows the CVE Search interface. At the top, there's a search bar with 'Buffer overflow' entered and a 'Search' button. Below the search bar, a notice states that keyword searching is available and provides examples of valid search terms. The 'Search Results' section shows 17,327 results for 'Buffer overflow'. The results are displayed in a list, with the first three entries visible. Each entry includes a CVE ID, the CNA (MITRE Corporation), and a description of the vulnerability. The number '17,327' is highlighted with a red box.

CVE-2025-29363 CNA: MITRE Corporation
Tenda RX3 US_RX3V1.0br_V16.03.13.11_multi_TDE01 is vulnerable to buffer overflow via the schedStartTime and schedEndTime parameters at /goform/saveParentControllInfo. This vulnerability allows attackers to cause a Denial of Service (DoS) via a crafted packet.
[Show less](#)

CVE-2025-29362 CNA: MITRE Corporation
Tenda RX3 US_RX3V1.0br_V16.03.13.11_multi_TDE01 is vulnerable to Buffer Overflow via the list parameter at /goform/setPptpUserList. This vulnerability allows attackers to cause a Denial of Service (DoS) via a crafted packet.

CVE-2025-29361 CNA: MITRE Corporation
Tenda RX3 US_RX3V1.0br_V16.03.13.11_multi_TDE01 is vulnerable to Buffer Overflow via the list parameter at /goform/SetVirtualServerCfg. This vulnerability allows attackers to cause a Denial of Service (DoS) via a crafted

CVE-2025-26623

CNA: GitHub (maintainer security advisories)

Exiv2 is a C++ library and a command-line utility to read, write, delete and modify Exif, IPTC, XMP and ICC image metadata. A heap buffer overflow was found in Exiv2...

[Show more](#)**CVE-2025-21333**

CNA: Microsoft Corporation

Windows Hyper-V NT Kernel Integration VSP Elevation of Privilege Vulnerability

CVE-2025-0689

CNA: Red Hat, Inc.

When reading data from disk, the grub's UDF filesystem module utilizes the user controlled data length metadata to allocate its internal buffers. In certain scenarios, while iterating through disk sectors,...

[Show more](#)**CVE-2025-0395**

CNA: GNU C Library

When the assert() function in the GNU C Library versions 2.13 to 2.40 fails, it does not allocate enough space for the assertion failure message string and size information, which may lead to a buffer overflow if the message string size aligns to page size.

[Show less](#)**CVE-2025-0633**

CNA: Samsung TV & Appliance

Heap-based Buffer Overflow vulnerability in iniparser_dumpsection_ini() in iniparser allows attacker to read out of bound memory

CVE-2025-0611

CNA: Chrome

Object corruption in V8 in Google Chrome prior to 132.0.6834.110 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page. (Chromium security severity: High)

开源社区的安全问题

- 现代开源社区带来软件安全性风险
- 2024年XZ Utils供应链后门攻击（CVE-2024-3094）
- XZ Utils是一款广泛应用于Linux和macOS的开源数据压缩工具，其维护者自2009年起由单一开发者负责。
- 2022年起，攻击者以“Jia Tan”（GitHub账号JiaT75）身份参与项目贡献，通过长期提交代码逐步获得维护权限，最终在2024年2月发布的5.6.0版本中植入后门。
- Fedora 41、Debian测试版、Kali Linux等未稳定版本受波及
- 全球约3%的Linux设备面临完全控制风险

内核级Rootkit攻击-系统调用劫持

https://blogs.technet.microsoft.com/markrussinovich/2005/10/31/sony-rootkits-and-digital-rights-management-gone-too-far/
Mark Russinovich's technical blog covering topics such as Windows troubleshooting, technologies and security.

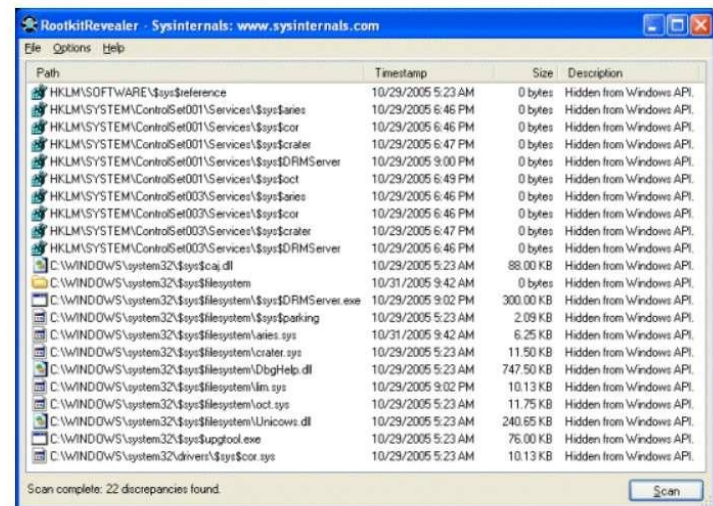
Sony, Rootkits and Digital Rights Management Gone Too Far

OttoHelweg2 October 31, 2005

Rate this article ★★★★★

0 0 8

Last week when I was testing the latest version of RootkitRevealer (RKR) I ran a scan on one of my systems and was shocked to see evidence of a rootkit. Rootkits are cloaking technologies that hide files, Registry keys, and other system objects from diagnostic and security software, and they are usually employed by malware attempting to keep their implementation hidden (see my "Unearthing Rootkits" article from the June issue of Windows IT Pro Magazine for more information on rootkits). The RKR results window reported a hidden directory, several hidden device drivers, and a hidden application:



Path	Timestamp	Size	Description
HKLM\SOFTWARE\sys\$reference	10/29/2005 5:23 AM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet001\Services\sys\$aries	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet001\Services\sys\$cor	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet001\Services\sys\$crater	10/29/2005 6:47 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet001\Services\sys\$DRMServer	10/29/2005 9:00 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet001\Services\sys\$oct	10/29/2005 6:49 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet003\Services\sys\$aries	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet003\Services\sys\$cor	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet003\Services\sys\$crater	10/29/2005 6:47 PM	0 bytes	Hidden from Windows API
HKLM\SYSTEM\ControlSet003\Services\sys\$DRMServer	10/29/2005 6:46 PM	0 bytes	Hidden from Windows API
C:\WINDOWS\system32\sys\$caj.dll	10/29/2005 5:23 AM	88.00 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem	10/31/2005 9:42 AM	0 bytes	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\sys\$DRMServer.exe	10/29/2005 9:02 PM	300.00 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\sys\$parking	10/29/2005 5:23 AM	2.09 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\aries.sys	10/31/2005 9:42 AM	6.25 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\crater.sys	10/29/2005 5:23 AM	11.50 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\dtbghelp.dll	10/29/2005 5:23 AM	747.50 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\lin.sys	10/29/2005 9:02 PM	10.13 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\loc.sys	10/29/2005 5:23 AM	11.75 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$filesystem\unicows.dll	10/29/2005 5:23 AM	240.65 KB	Hidden from Windows API
C:\WINDOWS\system32\sys\$upgtool.exe	10/29/2005 5:23 AM	76.00 KB	Hidden from Windows API
C:\WINDOWS\system32\drivers\sys\$cor.sys	10/29/2005 5:23 AM	10.13 KB	Hidden from Windows API

Scan complete: 22 discrepancies found.

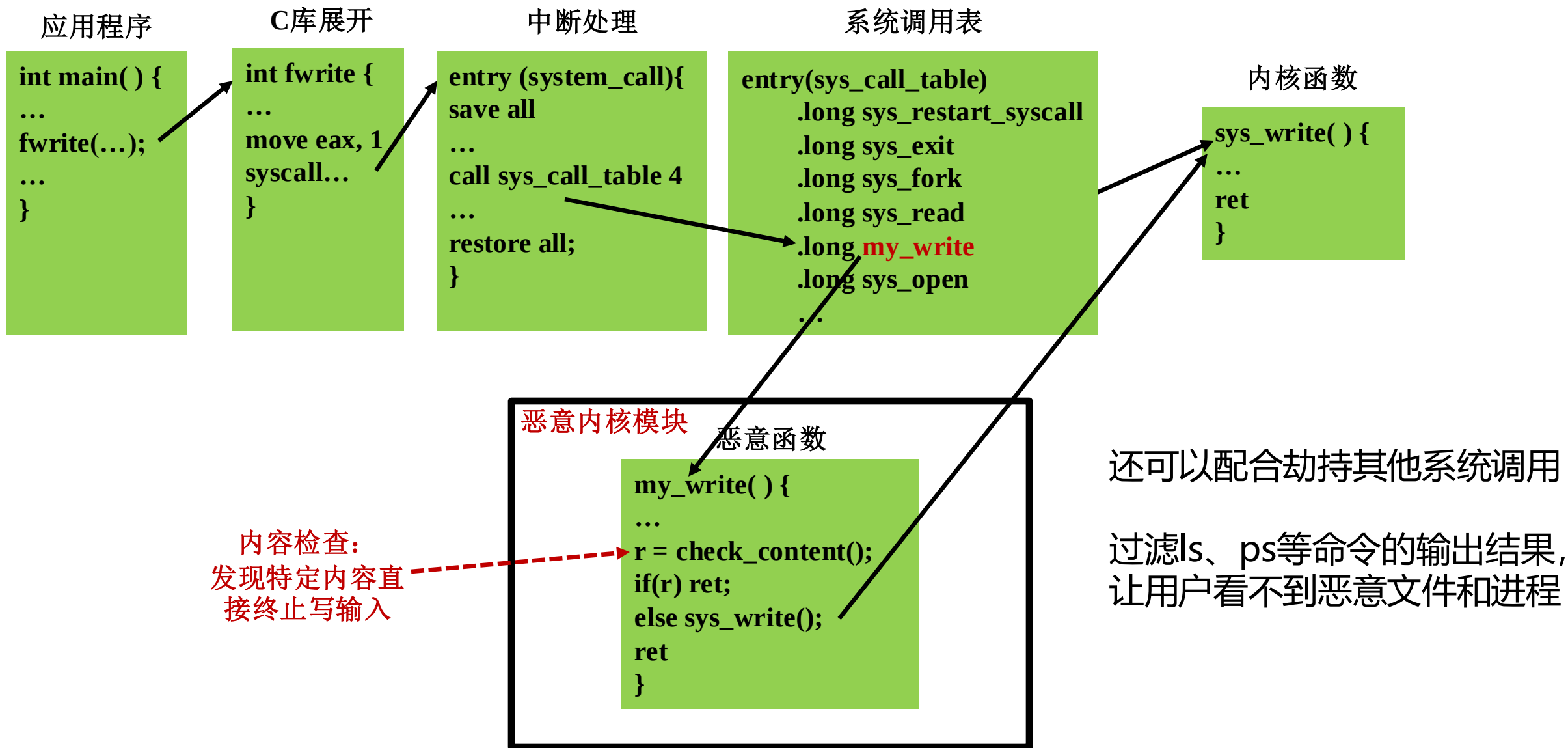
- 一位安全工程师在测试自己写的rootkit工具时，突然发现自己的电脑上有了rootkit。
- 他仔细研究之后发现，居然是前几天购买的CD悄悄地给系统上安装了rootkit，主要的目的是防止歌曲被拷贝，也即反盗版。

内核级rootkit一般通过恶意的内核模块劫持一部分系统调用

它是如何实现的呢？

知乎 @ustcsse308

内核级Rootkit攻击-代码注入



针对缓冲区溢出攻击，怎么做？

■ 避免溢出漏洞

- 使用系统级的防护
- 编译器使用“栈金丝雀”(stack canaries)

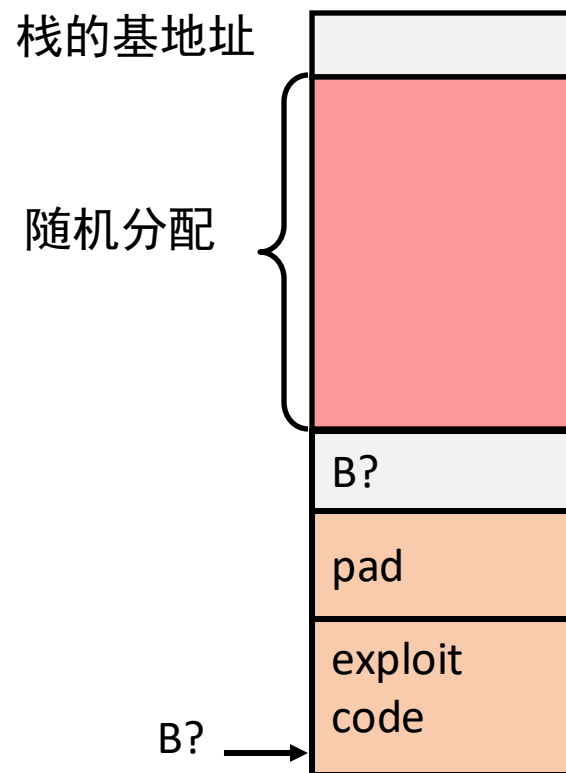
1. 代码中避免溢出漏洞(!)

```
/* Echo Line */  
void echo(){  
    char buf[4]; /* Way too small! */  
    fgets(buf, 4, stdin);  
    puts(buf);  
}
```

- 例如，使用**限制字符串**长度的库例程
 - **fgets** 代替 **gets**
 - strncpy 代替 strcpy (strcpy_s @ windows)
 - 在scanf函数中别用%s
 - 用fgets读入字符串
 - 或用 %ns代替%s, 其中n是一个合适的整数

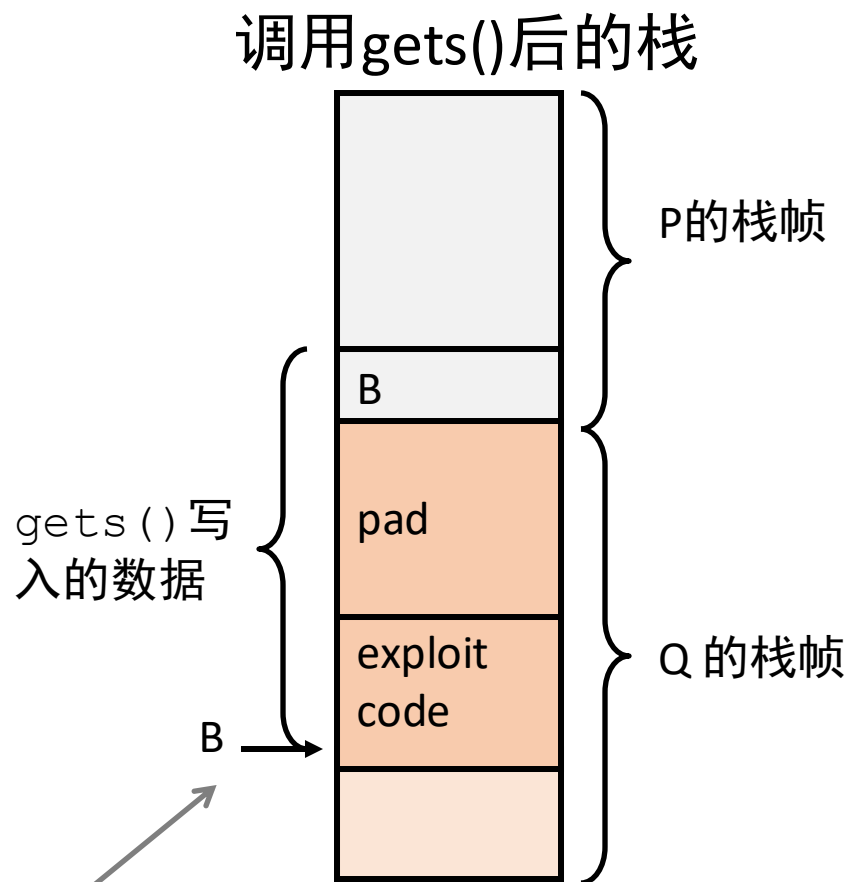
2. 系统级防护

- 随机的栈偏移
 - 程序启动后，在栈中分配随机数量的空间
 - 将移动整个程序使用的栈空间地址
 - 黑客很难预测插入代码的起始地址
 - 例如：执行5次内存申请代码
 - 每次程序执行，栈都重新定位



2. 系统级防护

- 非可执行代码段
 - 在传统的x86中，可以标记存储区为“只读”或“可写的”
 - 可以执行任何可读的操作
 - x86-64添加显式“执行”权限
 - 将stack标记为不可执行



所有执行该代码的尝试都将失败

3. 栈金丝雀(Stack Canaries)

- 想法
 - 在栈中buffer之后的位置放置特殊的值——**金丝雀** (“canary”)
 - 退出函数之前，检查是否被破坏
- 用GCC 实现
 - **-fstack-protector**
 - 该选项现在是默认开启的(早期默认关闭)

```
unix> ./bufdemo-sp  
Type a string: 0123456  
0123456
```

```
unix> ./bufdemo-sp  
Type a string: 01234567  
*** stack smashing detected ***
```

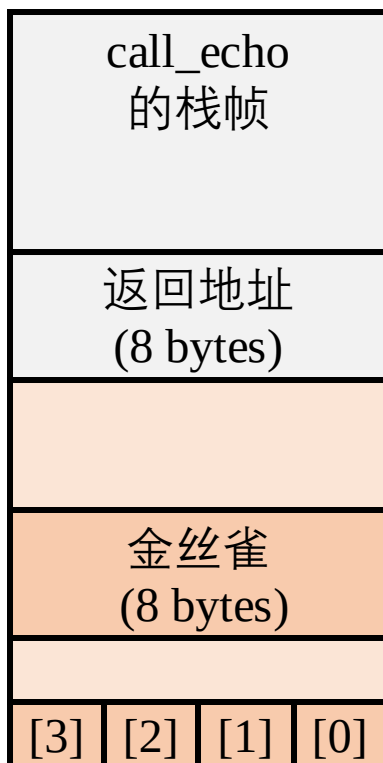
保护缓冲区反汇编

echo:

```
40072f:    sub    $0x18,%rsp
400733:    mov     %fs:0x28,%rax
40073c:    mov     %rax,0x8(%rsp) #放置 (段寻址)
400741:    xor     %eax,%eax
400743:    mov     %rsp,%rdi
400746:    callq   4006e0 <gets>
40074b:    mov     %rsp,%rdi
40074e:    callq   400570 <puts@plt>
400753:    mov     0x8(%rsp),%rax
400758:    xor     %fs:0x28,%rax    #检测
400761:    je      400768 <echo+0x39>
400763:    callq   400580 <__stack_chk_fail@plt>
400768:    add     $0x18,%rsp
40076c:    retq
```

设立金丝雀(Canary)

调用gets之前

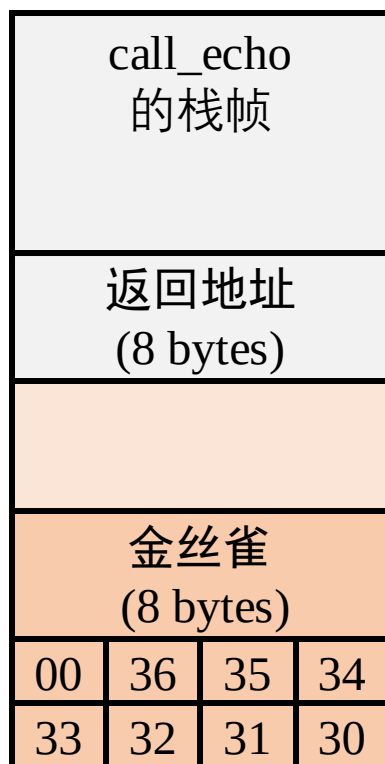


```
/* Echo Line */  
void echo(){  
    char buf[4]; /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

```
echo:  
    ...  
    movq    %fs:40, %rax # Get canary  
    movq    %rax, 8(%rsp) # Place on stack  
    xorl    %eax, %eax  # Erase canary  
    ...
```

核对金丝雀

调用gets后



```
/* Echo Line */
void echo(){
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
    ...
    movq    8(%rsp), %rax    # Retrieve from stack
    xorq    %fs:40, %rax    # Compare to canary
    je      .L6             # If same, OK
    call     __stack_chk_fail # FAIL
.L6:
    ...
```

buf ← %rsp

Input: 0123456

面向返回的编程攻击

- 挑战(对黑客)
 - 栈随机化使缓冲区位置难以预测。
 - 标记栈为不可执行，很难插入二进制代码
- 替代策略
 - 使用已有代码
 - 例如：stdlib的库代码
 - 将片段串在一起以获得总体期望的结果。
 - 无需克服栈金丝雀
- 从小工具构建攻击程序
 - 以**ret**结尾的指令序列
 - 单字节编码为0xc3
 - 每次运行，代码的位置固定
 - 代码可执行

如何破解金丝雀？

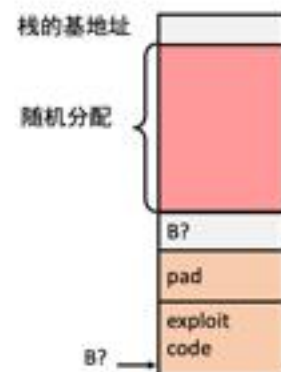
- 金丝雀总是以0x00开头
- 对于32位的程序而言，金丝雀一共四个字节
 - 也就是说还有3个字节不可知
- 如何获取这三个字节？
- 挑战：每个函数的金丝雀值都会变化
 - 检测到金丝雀值错误，程序就会崩溃，因此只有一次尝试的机会！
- 方法：利用fork机制，变出无穷次尝试机会！
 - fork机制会建立一个新的进程，同时保持内存数据不变（包括金丝雀）
 - 在fork出来的进程中循环暴力搜索金丝雀的值
 - 即使猜错了崩溃的也只是子进程，与父进程无关，可以继续fork继续猜
 - 直到猜出正确的金丝雀的值

基于随机的保护机制在fork下几乎都失效

2. 系统级防护

• 随机的栈偏移

- 程序启动后，在栈中分配随机数量的空间
- 将移动整个程序使用的栈空间地址
- 黑客很难预测插入代码的起始地址
- 例如：执行5次内存申请代码
 - 每次程序执行，栈都重新定位



核对金丝雀

调用gets后



```
/* Echo Line */
void echo(){
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
    . . .
    movq    8(%rsp), %rax    # Retrieve from
stack
    xorq    %fs:40, %rax    # Compare to canary
    je      .L6             # If same, OK
    call    __stack_chk_fail # FAIL
.L6:
    . . .
```

Input: 0123456

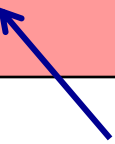
nop sled 空操作雪橇

- 在实际攻击代码之前，插入很长一段nop指令
- nop除了对程序计数器加一没有任何效果
- 只要猜对这段序列中某一个指令的地址，指令就会沿着序列划过，最终执行攻击代码。
- 这样，以枚举的方式就可以破解栈随机化。

小工具例子 #1 (假设栈中返回地址被修改成0x4004d4)

```
long ab_plus_c(long a, long b, long c)
{
    return a*b + c;
}
```

```
00000000004004d0 <ab_plus_c>:
4004d0: 48 0f af fe      imul %rsi,%rdi
4004d4: 48 8d 04 17      lea (%rdi,%rdx,1),%rax
4004d8: c3               retq
```

 $\text{rax} \leftarrow \text{rdi} + \text{rdx}$

小工具地址 = 0x4004d4

- 使用现有功能的尾部

小工具例子 #2 (假设栈中返回地址被修改成0x4004dc)

```
void setval(unsigned *p) {  
    *p = 3347663060u;  
}
```

movq %rax, %rdi的编码

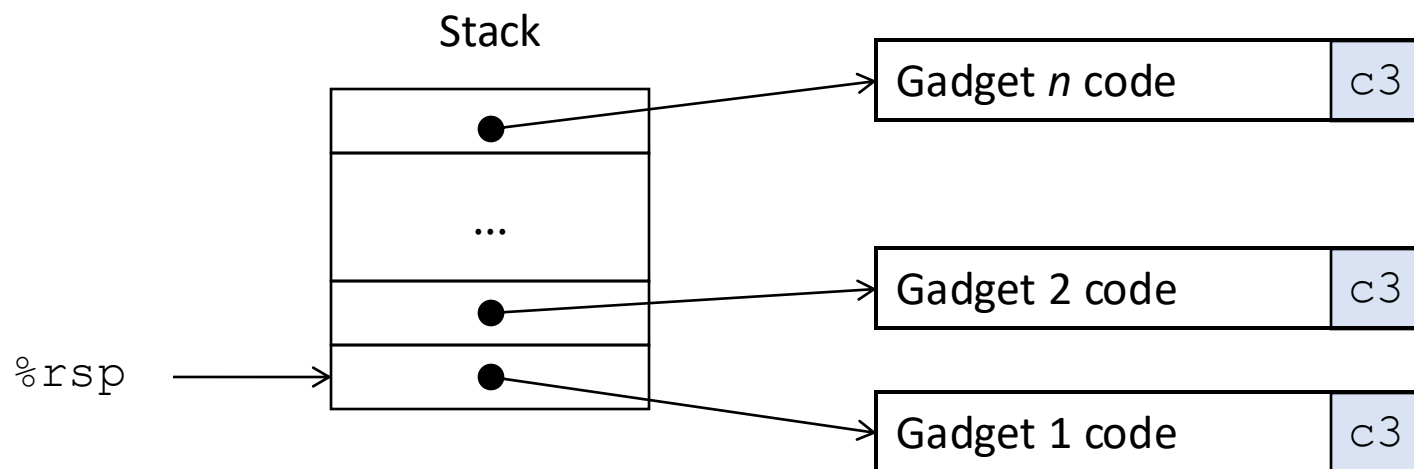
<setval>:
4004d9: c7 07 d4 48 89 c7 movl \$0xc78948d4,(%rdi)
4004df: c3 retq

rdi ← rax

小工具地址 = 0x4004dc

- 改变字节码的用途

面向返回编程(ROP)的 执行



- `ret` 指令触发
 - 将开始运行 Gadget 1
- 每个小工具最终的 `ret` 将启动下一个小工具
- 通过小工具序列的运行，达到攻击目的。

存在安全隐患的缓冲区代码

```
/* Echo Line */  
void echo()  
{  
    char buf[4]; /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

问题：分配多大才足够？

```
void call_echo() {  
    echo();  
}
```

```
unix> ./bufdemo-nsp  
Type a string: 012345678901234567890123  
012345678901234567890123
```

```
unix> ./bufdemo-nsp  
Type a string: 0123456789012345678901234  
Segmentation Fault
```


主要内容

- 内存布局
- 缓冲区溢出
 - 安全隐患
 - 防护
- 联合

联合union

- 一个联合的总大小等于最大的字段大小。
- 可以规避c语言的类型系统，但也会产生很多讨厌的错误

例如：

一个二叉树，只有叶子节点有两个数据，内部节点没有数据。

```
union node_u{
    struct{
        union node_u *left;
        union node_u *right;
    }internal;
    double data[2];
}
```

大小只有16字节。

如果n是指向节点的指针：n->internal.left和&data[0]指向同一块存储空间。

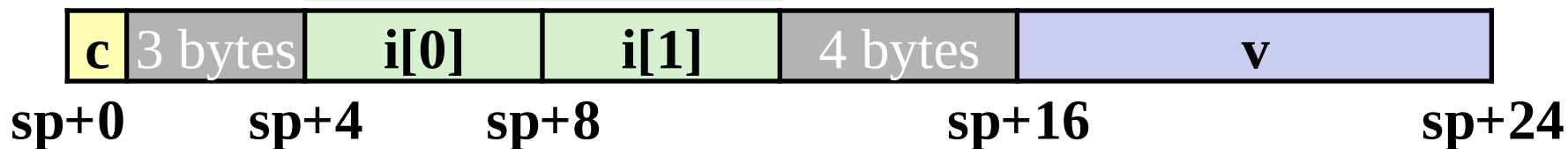
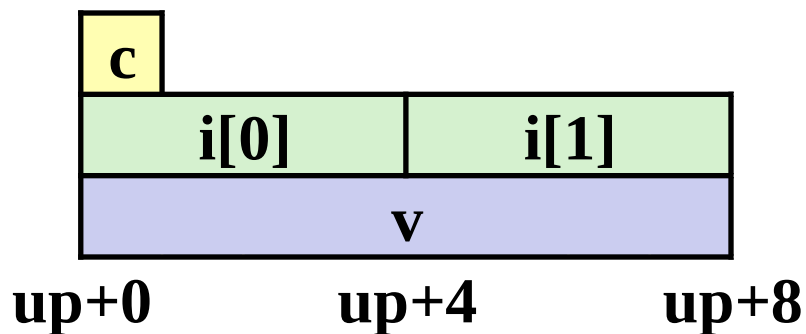
联合的内存分配

- 依据最大成员申请内存

同时只能使用一个成员

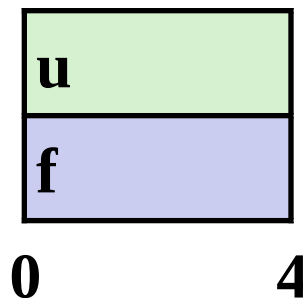
```
union U1 {  
    char c;  
    int i[2];  
    double v;  
} *up;
```

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *sp;
```



使用联合获取位模式 (重新解读)

```
typedef union {  
    float f;  
    unsigned u;  
} bit_float_t;
```



```
float bit2float(unsigned u) {  
    bit_float_t arg;  
    arg.u = u;  
    return arg.f;  
}
```

是否和(float)u 相同？

```
unsigned float2bit(float f) {  
    bit_float_t arg;  
    arg.f = f;  
    return arg.u;  
}
```

是否和(unsigned)f 相同？

字节序

- 想法

- short/long/quad words 在内存中用连续的2/4/8 字节存储
- 哪个字节是最高/低位?
- 在不同机器之间交换顺序, 会有问题。

- 大端序(Big Endian)

- 最高有效位在低地址, 如sun 的工作站Sparc

- 小端序(Little Endian)

- 最低有效位在低地址, 如Intel x86、ARM Android、IOS

- 双端序(Bi Endian)

- 可配置成大/小端序, 如ARM

字节序的例子

```
union {  
    unsigned char c[8];  
    unsigned short s[4];  
    unsigned int i[2];  
    unsigned long l[1];  
} dw;
```

32-bit

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

64-bit

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

字节序的例子(续...)

```
int j;
for (j = 0; j < 8; j++)
    dw.c[j] = 0xf0 + j;

printf("Characters 0-7 == [0x%x,0x%x,0x%x,0x%x,0x%x, "
       "0x%x,0x%x,0x%x]\n",
       dw.c[0], dw.c[1], dw.c[2], dw.c[3],
       dw.c[4], dw.c[5], dw.c[6], dw.c[7]);

printf("Shorts 0-3 == [0x%x,0x%x,0x%x,0x%x]\n",
       dw.s[0], dw.s[1], dw.s[2], dw.s[3]);

printf("Ints 0-1 == [0x%x,0x%x]\n",
       dw.i[0], dw.i[1]);

printf("Long 0 == [0x%lx]\n",
       dw.l[0]);
```

IA32的字节序

小端序

f0	f1	f2	f3	f4	f5	f6	f7
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

LSB ← Print → MSB LSB MSB

输出:

Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]

Shorts 0-3 == [0xf1f0,0xf3f2,0xf5f4,0xf7f6]

```
Ints    0-1 == [0xf3f2f1f0,0xf7f6f5f4]
```

Long 0 == [0xf3f2f1f0]

Sun的字节序

大端序

f0	f1	f2	f3	f4	f5	f6	f7
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

MSB



LSB

MSB

LSB

Print

Sun机器的输出:

Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]

Shorts 0-3 == [0xf0f1,0xf2f3,0xf4f5,0xf6f7]

Ints 0-1 == [0xf0f1f2f3,0xf4f5f6f7]

Long 0 == [0xf0f1f2f3]

x86-64的字节序

小端序

f0	f1	f2	f3	f4	f5	f6	f7
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

LSB

MSB

Print

x86-64机器的输出

Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]

Shorts 0-3 == [0xf1f0,0xf3f2,0xf5f4,0xf7f6]

Ints 0-1 == [0xf3f2f1f0,0xf7f6f5f4]

Long 0 == [0xf7f6f5f4f3f2f1f0]

经典例题

1.简述缓冲区溢出攻击的原理以及防范方法（5分）。

攻击原理（3个采分点）：

- （1）程序输入缓冲区写入特定的数据，例如在gets读入字符串时
- （2）使位于栈中的缓冲区数据溢出，用特定的内容覆盖栈中的内容，例如函数返回地址等
- （3）使得程序在读入字符串，结束函数gets从栈中读取返回地址时，错误地返回到特定的位置，执行特定的代码，达到攻击的目的。

防范方法(2个采分点):

- （1）代码中避免溢出漏洞：例如使用限制字符串长度的库函数。
- （2）随机栈偏移：程序启动后，在栈中分配随机数量的空间，将移动整个程序使用的栈空间地址。
- （3）限制可执行代码的区域
- （4）进行栈破坏检查——金丝雀

补充解释： 对抗缓冲区溢出攻击4种方法

1) 使用限制字符串长度的库函数。

2) 栈随机化

栈随机化的思想使得栈的位置在程序每次运行时都有变化。程序开始时，在栈上分配一段0~n字节之间的随机大小的空间。分配的范围n必须足够大，才能获得足够多的栈地址变化，但是又要足够小，不至于浪费程序太多空间。

Linux系统中，栈随机化已经变成了标准行为。是更大的一类技术中的一种，这类技术称为地址空间布局随机化。每次运行时程序的不同部分，包括程序代码、库代码、栈、全局变量和堆数据，都会被加载到内存的不同区域。

3) 限制可执行代码区域

消除攻击者向系统中插入可执行代码的能力，限制哪些内存区域能够存放可执行代码，只有保存编译器产生的代码的那部分才需要是可执行的，其他部分可以被限制为只允许读和写。

4) 栈破坏检测

最近的GCC版本在产生的代码中加入了一种栈保护者机制，来检测缓冲区越界。其思想是在栈帧中任何局部缓冲区与栈状态之间存储一个特殊的值，也称哨兵值，是在程序每次运行时随机产生的。在恢复寄存器状态和从函数返回前，程序检查这个金丝雀值是否被该函数的某个操作或者该函数调用的某个函数的某个操作改变了。

经典例题

2. 简述C编译过程对非寄存器实现的int全局变量与非静态int局部变量处理的区别。包括存储区域、赋初值、生命周期、指令中寻址方式等。

	int全局变量	int局部变量
存储区域	数据段	堆栈段
赋初值	编译时 int x=1;	程序执行时，执行数据传送类指令如 MOVL \$1234, 8(RSP)
生命周期	程序整个执行过程中都存在	进入子程序后在堆栈中存在(如执行subq \$8, %rsp)子程序返回前清除消失
指令中寻址方式	其地址是个常数, 寻址如 movl 0x806808C, %eax	通过rsp/rbp的寄存器相对寻址方式。如类似 (%rsp) 或8(%rsp)或-8(%rbp)等

经典例题

3.当调用malloc这样的C标准库函数时，()可以在运行时动态的扩展和收缩。

A. 堆 B. 栈 C. 共享库 D. 内核虚拟存储器

答案: **A** 考点: malloc如何动态分配内存地

4.当函数调用时，()可以在程序运行时动态地扩展和收缩。

A.程序代码和数据区 B. 栈 C. 共享库 D. 内核虚拟存储器

答案: **B** 考点: 栈在程序运行时的状态

C语言复合类型总结

- 数组
 - 连续分配内存
 - 对齐：满足每个元素对齐要求
 - 数组名是首个元素的指针常量
 - 没有越界检查！
- 结构体
 - 各成员按结构体定义中的顺序分配内容
 - 在中间、末尾填充字节，以满足对齐要求
- 联合
 - 覆盖的声明
 - 规避类型系统对编程束缚的方法

Enjoy!